



HACKYEAH 2026 · HUAWEI · IMAGINE WHAT'S NEXT

Aegis Pocket

Approve AI Agents

A native HarmonyOS app that puts a human back in the loop when an AI agent wants to do something risky.

ArkTS + ArkUI

API 20+ · target API 24

Human-Centric

Intelligent

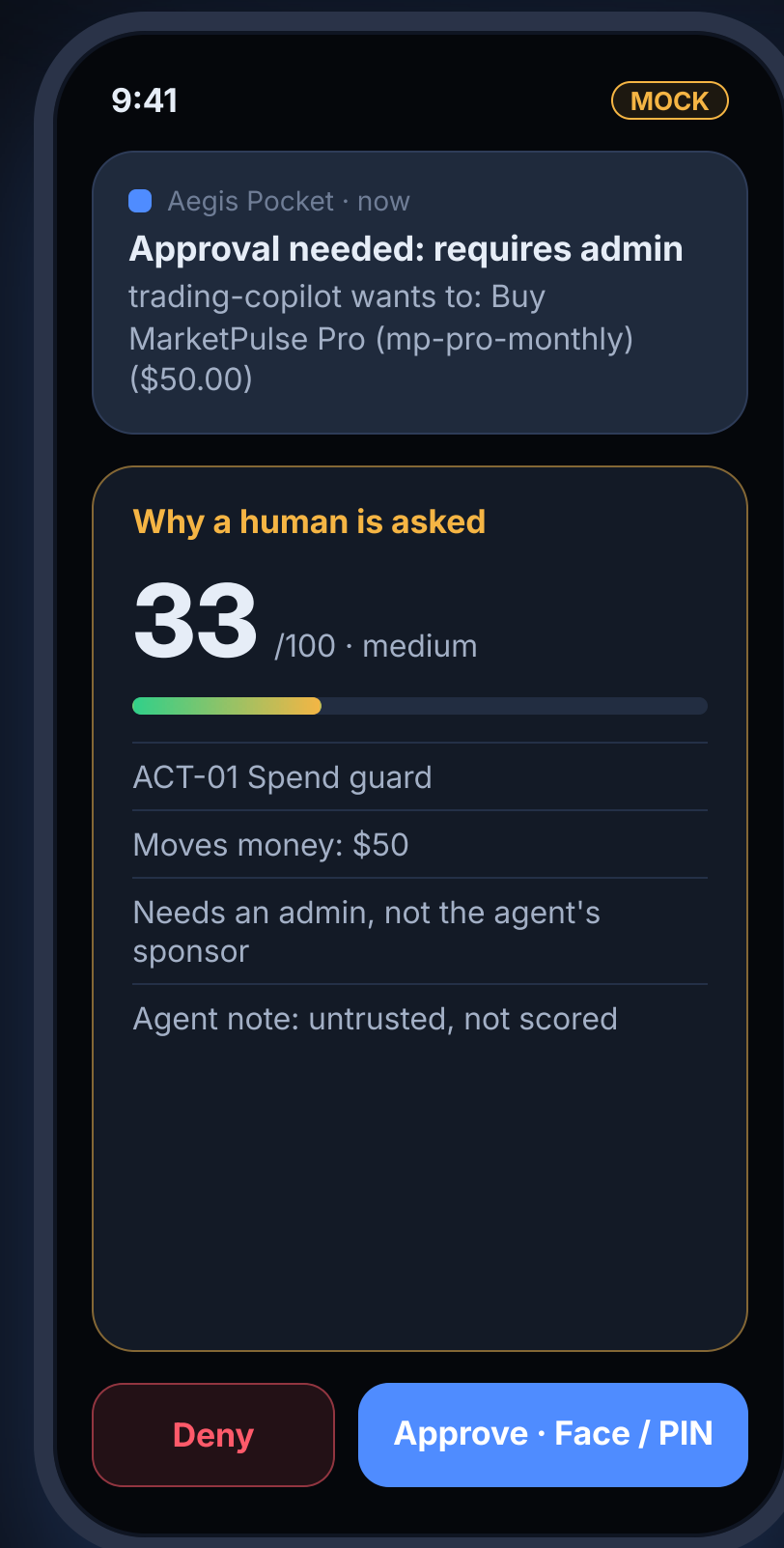


Illustration in HTML/CSS, not an app screenshot

THE PROBLEM

Agents act for us. Humans are out of the loop.

AI agents now buy subscriptions, read company databases, deploy code and email customers. When one wants to do something risky, someone has to say yes or no, quickly and safely, and that person is rarely at a desk.

Spend

"Research agent wants to spend \$50 on a SaaS subscription." Who approves it, and on what evidence?

Data

"Read the customers table (contains PII)." Personal data can leave the organisation in one tool call.

Authority

The agent's sponsor should not approve its own large spend. Roles and separation of duties live in a web console nobody watches.

Manipulation

The agent explains itself, but that text can be prompt-injected. Approvers need facts, not the agent's sales pitch.

WHAT AN APPROVER NEEDS

Reach

where they are: the phone

Context

facts, not the agent's pitch

Authority

the right role decides

Assurance

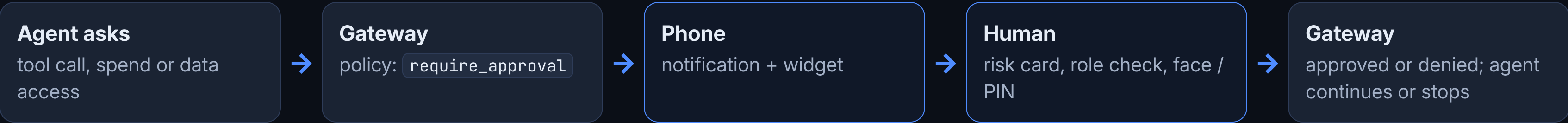
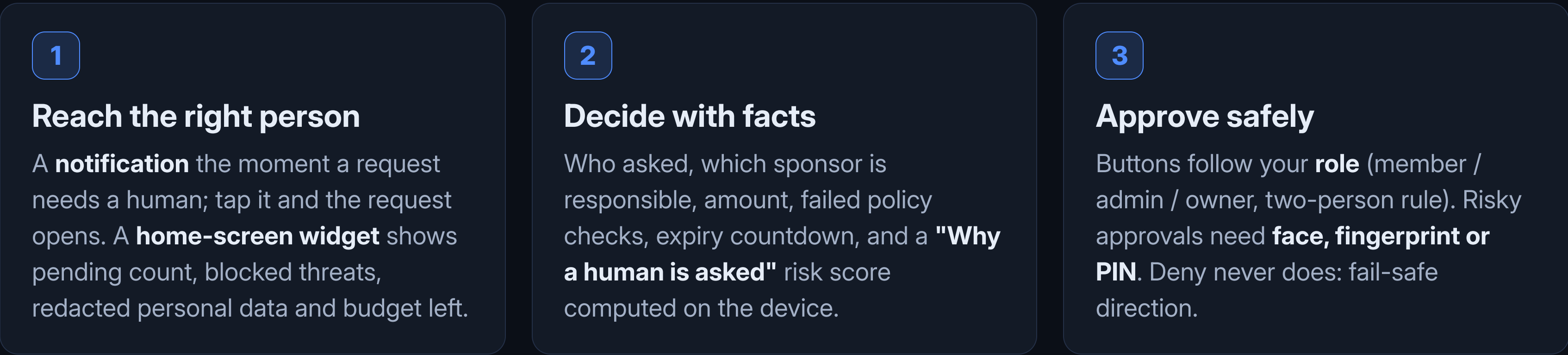
it is really them

Result: approvals either block the agent for hours or get rubber-stamped. Neither is human oversight.

THE SOLUTION

The approval desk moves to your phone

Aegis Pocket is the human-in-the-loop front end for the Aegis gateway (our separate AI guardrails project). The gateway decides *that* a human must be asked; the phone makes asking fast, informed and safe.



Mock mode (default, labelled **MOCK** in the header) simulates the gateway on the device, so the app demos without a server. **Live mode** talks to the real gateway over HTTP.

Three things the phone does better than a console

Notification → step-up approve

1. New pending request → **Notification Kit** notification + in-app banner + haptic cue.
2. Tap → WantAgent deep link → `onNewWant` opens the request detail.
3. **Approve** on admin / owner / two-person requests or anything $\geq \$100$ → system face / fingerprint / PIN sheet (ATL3, 32-byte random challenge).
4. No authenticator (e.g. emulator) → dialog *"Device authentication unavailable"* with a labelled manual fallback.

Role-locked approvals

1. Personas: Piotr (member), Emily / Marek (admin), Katarzyna (owner).
2. \$50 subscription needs an admin; \$480 seats need owner + admin; budget raise \$60 → \$150 needs the owner; reading the PII table needs an admin.
3. Otherwise Approve shows **LOCKED** with the reason: *"Requires owner (you are admin)"*, *"Separation of duties"*.
4. Optimistic vote; rolled back with the gateway's reason on 403 / 409.

Risk explainer

1. Deterministic 0-100 score from the gateway's policy data: control that fired, amount, budget increase, required role, production, sensitive data, data egress, failed checks.
2. Plain-language reasons; levels low / medium (≥ 30) / high (≥ 60).
3. The agent's own note is shown, labelled **agent-supplied, untrusted**, and **never scored**.
4. Rule-based on the device: no model, no network.

Built on HarmonyOS Kits, not a web view

CAPABILITY	KIT / API	WHERE IN CODE (ENTRY/SRC/MAIN/ETS/)
Notifications + tap-to-open	Notification Kit <code>publish</code> , <code>requestEnableNotification</code> ; Ability Kit <code>wantAgent</code>	<code>platform/NotificationService.ets</code> · <code>entryability/EntryAbility.ets</code>
Service widgets 2×2 and 2×4	Form Kit <code>FormExtensionAbility</code> , <code>formProvider.updateForm</code> , <code>postCardAction</code>	<code>formability/PostureFormAbility.ets</code> · <code>widget/pages/*</code> · <code>platform/WidgetBridge.ets</code>
Step-up before risky approvals	User Authentication Kit <code>getAvailableStatus</code> , <code>getUserAuthInstance</code> (face / fingerprint / PIN, ATL3); Crypto <code>createRandom</code>	<code>platform/StepUpAuth.ets</code> · <code>views/DetailView.ets</code>
Haptics on approve / deny / new request	Sensor Service Kit <code>vibrator.startVibration</code> (preset <code>haptic.notice.*</code>)	<code>platform/Haptics.ets</code>
Background refresh while closed	Background Tasks Kit <code>workScheduler.startWork</code> , <code>WorkSchedulerExtensionAbility</code> (network-gated, ~2 h)	<code>platform/BackgroundRefresh.ets</code> · <code>workscheduler/RefreshWorkAbility.ets</code>
Live mode to the gateway	Network Kit <code>http.createHttp().request</code> , header <code>X-Aegis-View-As</code>	<code>data/LiveRepository.ets</code>
Settings shared with the widget	ArkData Preferences	<code>platform/SettingsStore.ets</code> · <code>platform/WidgetBridge.ets</code>
UI + accessibility	ArkUI <code>Navigation</code> , <code>Tabs</code> , animations; <code>accessibilityText</code> , <code>fp</code> fonts, <code>minHeight</code> buttons	<code>pages/Index.ets</code> · <code>views/*</code> · <code>components/Common.ets</code>

Permissions: `INTERNET`, `ACCESS_BIOMETRIC`, `VIBRATE` (all `system_grant`, normal APL: an ordinary app, no system privileges).

See what an AI model would receive, before it is sent

What you typed

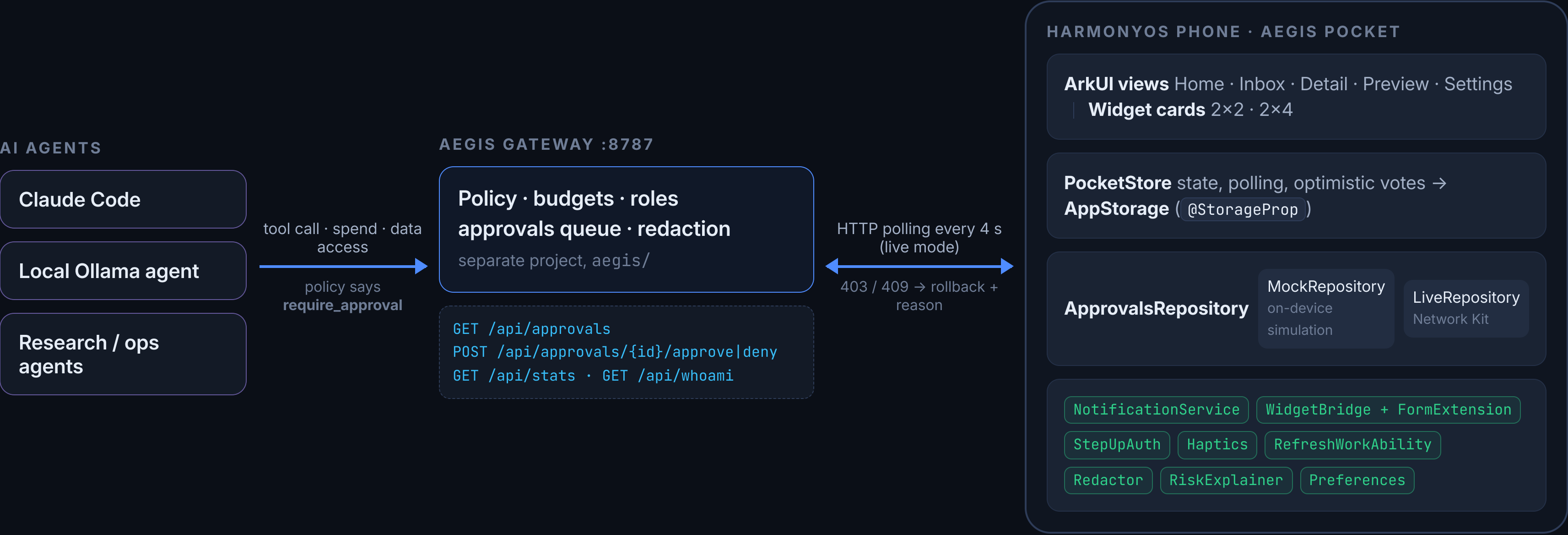
Hi, please summarise the complaint from Anna Kowalska (~~anna.kowalska@example.com~~, tel. ~~+48-601-234-567~~). Her PESEL is ~~44051401359~~ and the refund should go to ~~PL61-1090-1014-0000-0712-1981-2874~~. She paid with card ~~4111-1111-1111-1111~~.

What leaves the device

Hi, please summarise the complaint from Anna Kowalska ([EMAIL_1], tel. [PHONE_1]). Her PESEL is [PESEL_1] and the refund should go to [IBAN_1]. She paid with card [PAN_1].

- **Validators, not bare regex:** PESEL weights 1-3-7-9 + control digit, Luhn for card numbers, IBAN mod-97. Invalid numbers are left alone.
- **Runs in ArkTS on the device:** no network, nothing stored; logs never contain payloads. Findings list shows masked values only (card: first 6 / last 4).
- **Same contract as the gateway:** entity names and stable [ENTITY_N] placeholders mirror Aegis's local-first redaction. Names are not detected (no NER model on the phone).

Agents → gateway → phone, and back



Views never see the wire format: gateway types (`model/Wire.ets`) are mapped to UI models in `LiveRepository.map`. Widgets read a Preferences snapshot pushed by the app, by their own 30-min update, and by the WorkScheduler task while the app is closed.

60 seconds on the emulator

Live demo on HarmonyOS 6.1.1 (API 24) emulator

[see video](#)

Screenshot slot: add PNGs to docs/screenshots/ and run docs/deck/build.sh

1. **Widget** on the home screen (MOCK badge); tap opens the app.
2. **Home**: pending count, threats blocked, data redacted, budget left.
3. Simulated request arrives: **notification** + banner; tap it.
4. **Detail**: "Why a human is asked" score, facts, checks, untrusted agent note.
5. **Inbox**: switch Emily (admin) → Piotr (member); Approve becomes **LOCKED**.
6. As Emily: **Approve** → step-up sheet (or labelled fallback on the emulator) → approved.
7. **Preview**: [EMAIL_1], [PESEL_1] placeholders.

Recorded in **mock mode** (label visible). The DevEco emulator cannot simulate biometrics or vibration, so the video shows the PIN or the labelled fallback, and no haptics.

From a clean checkout to a running app

```
git clone https://github.com/JustAnotherDevv/tiles-hackyeah-2026
cd tiles-hackyeah-2026 && ohpm install
```

```
# build (DevEco Studio 6.1.1, devecocli 1.3.4)
devecocli build --modules entry --build-mode debug
```

```
# sign: DevEco > Project Structure > Signing Configs
#       (your own Huawei ID; keys stay outside the repo)
HAP=entry/build/default/outputs/default/entry-default-signed.hap
hdc install -r $HAP
hdc shell aa start -a EntryAbility -b com.hackyeah.aegispocket
hdc hilog | grep AegisPocket
```

```
# 17 local unit tests: redactor, eligibility,
# step-up rule, risk explainer (README section 8)
hvigow --mode module -p module=entry@default test
```

- **No server needed:** mock mode is the default; live mode needs only the gateway URL in Settings (`http://10.0.2.2:8787` on the emulator).
- **Checked by us:** build exit 0, lint 0 errors, 17 / 17 unit tests pass. The same source also builds against the **OpenHarmony 6.1 SDK** (API 23) with no stubs.
- **Honest limits:** biometrics and vibration need a real device; background refresh is system-scheduled (~2 h), a safety net, not real-time. See README "Known limitations".
- **Public-safe:** empty `signingConfigs`, no keys or tokens; all mock people and amounts are fictional.
- **AI disclosure:** built with Claude Code (Opus 5.5) and the organizers' HarmonyOS skills; logged in `AI_WORKFLOW.md`. The app itself contains no AI model.

WHY IT MATTERS

Autonomy for agents. Authority for people.

Human-Centric

Oversight of autonomous AI where people actually are: on the phone, with role rules, separation of duties and accessible controls.

Intelligent

On-device risk explanation that refuses to trust the agent's own words, plus local PII detection with checksums.

Native

Notification Kit, Form Kit, User Authentication Kit, vibrator, WorkScheduler, Network Kit, ArkData, ArkUI. Not portable as-is to another OS.

Next

Push Kit or SSE for instant delivery while closed · wearable target for approve-from-the-wrist · verification on a physical device with biometrics.

REPOSITORY

github.com/JustAnotherDevv/tiles-hackyeah-2026

TEAM

JustAnotherDevv